

# 4

## THE MICROARCHITECTURE LEVEL

The level above the digital logic level is the microarchitecture level. Its job is to implement the ISA (Instruction Set Architecture) level above it, as illustrated in Fig. 1-2. The design of the microarchitecture level depends on the ISA being implemented, as well as the cost and performance goals of the computer. Many modern ISAs, particularly RISC designs, have simple instructions that can usually be executed in a single clock cycle. More complex ISAs, such as the Pentium 4, may require many cycles to execute a single instruction. Executing an instruction may require locating the operands in memory, reading them, and storing results back into memory. The sequencing of operations within a single instruction often leads to a different approach to control than that for simple ISAs.

### 4.1 AN EXAMPLE MICROARCHITECTURE

Ideally, we would like to introduce this subject by explaining the general principles of microarchitecture design. Unfortunately, there are no general principles; every one is a special case. Consequently, we will discuss a detailed example instead. For our example ISA, we have chosen a subset of the Java Virtual Machine, as we promised in Chap. 1. This subset contains only integer instructions, so we have named it **IJVM**. We will discuss the full JVM in Chap. 5.

We will start out by describing the microarchitecture on top of which we will implement IJVM. IJVM has some complex instructions. Many such architectures

have often been implemented through microprogramming, as discussed in Chap. 1. Although IJVM is small, it is a good starting point for describing the control and sequencing of instructions.

Our microarchitecture will contain a microprogram (in ROM), whose job is to fetch, decode, and execute IJVM instructions. We cannot use the Sun JVM interpreter for the microprogram because we need a tiny microprogram that drives the individual gates in the actual hardware efficiently. In contrast, the Sun JVM interpreter was written in C for portability, and cannot control the hardware at the level of detail we need. Since the actual hardware used consists only of the basic components described in Chap. 3, in theory, after fully understanding this chapter, the reader should be able to go out and buy a large bag full of transistors and build this subset of the JVM machine. Students who successfully accomplish this task will be given extra credit (and a complete psychiatric examination).

A convenient model for the design of the microarchitecture is to think of the design as a programming problem, where each instruction at the ISA level is a function to be called by a master program. In this model, the master program is a simple, endless loop that determines a function to be invoked, calls the function, then starts over, very much like Fig. 2-3.

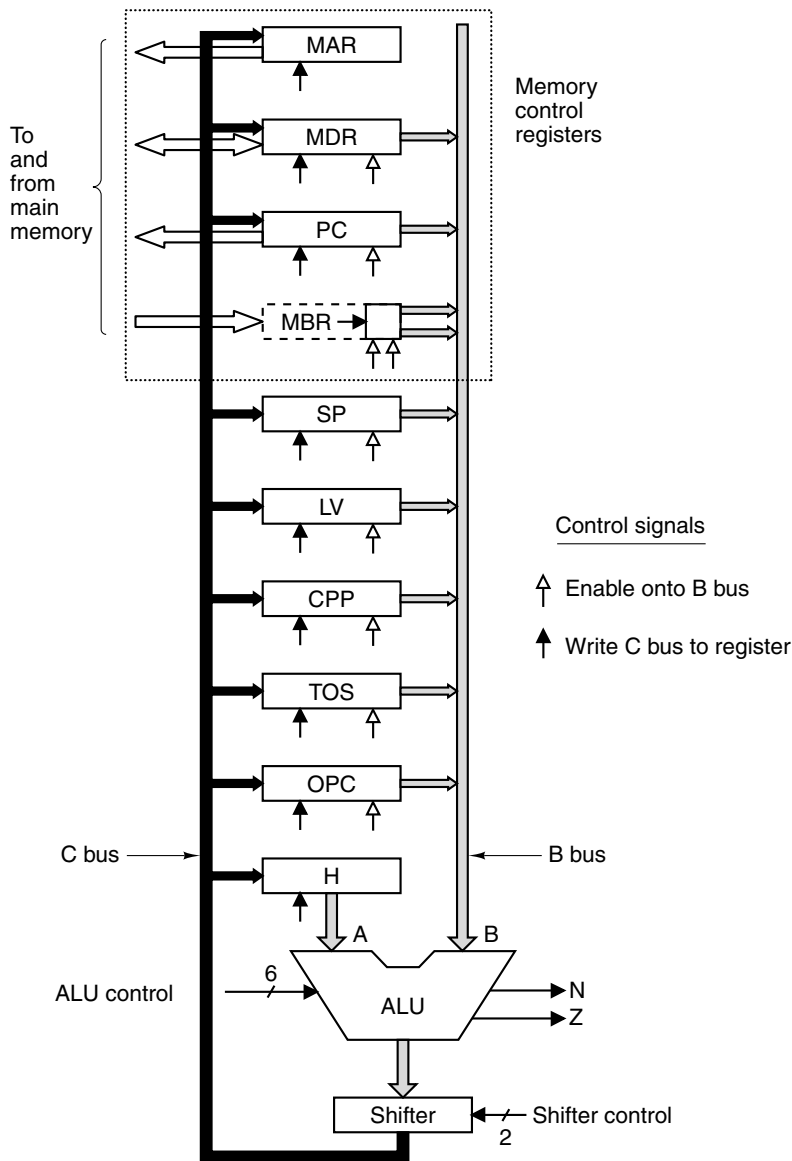
The microprogram has a set of variables, called the **state** of the computer, which can be accessed by all the functions. Each function changes at least some of the variables making up the state. For example, the Program Counter (PC) is part of the state. It indicates the memory location containing the next function (i.e., ISA instruction) to be executed. During the execution of each instruction, the PC is advanced to point to the next instruction to be executed.

IJVM instructions are short and sweet. Each instruction has a few fields, usually one or two, each of which has some specific purpose. The first field of every instruction is the **opcode** (short for **operation code**), which identifies the instruction, telling whether it is an ADD or a BRANCH, or something else. Many instructions have an additional field, which specifies the operand. For example, instructions that access a local variable need a field to tell *which* variable.

This model of execution, sometimes called the **fetch-execute cycle**, is useful in the abstract and may also be the basis for implementation for ISAs like IJVM that have complex instructions. Below we will describe how it works, what the microarchitecture looks like, and how it is controlled by the microinstructions, each of which controls the data path for one cycle. Together, the list of microinstructions forms the microprogram, which we will present and discuss in detail.

### 4.1.1 The Data Path

The **data path** is that part of the CPU containing the ALU, its inputs, and its outputs. The data path of our example microarchitecture is shown in Fig. 4-1. While it has been carefully optimized for interpreting IJVM programs, it is fairly similar to the data path used in most machines. It contains a number of 32-bit



**Figure 4-1.** The data path of the example microarchitecture used in this chapter.

registers, to which we have assigned symbolic names such as PC, SP, and MDR. Though some of these names are familiar, it is important to understand that these registers are accessible only at the microarchitecture level (by the microprogram). They are given these names because they usually hold a value corresponding to the variable of the same name in the ISA level architecture. Most registers can

drive their contents onto the B bus. The output of the ALU drives the shifter and then the C bus, whose value can be written into one or more registers at the same time. There is no A bus for the moment; we will add one later.

The ALU is identical to the one shown in Fig. 3-19 and Fig. 3-20. Its function is determined by six control lines. The short diagonal line labeled “6” in Fig. 4-1 indicates that there are six ALU control lines. These are  $F_0$  and  $F_1$  for determining the ALU operation, ENA and ENB for individually enabling the inputs, INVA for inverting the left input, and INC for forcing a carry into the low-order bit, effectively adding 1 to the result. However, not all 64 combinations of ALU control lines do something useful.

Some of the more interesting combinations are shown in Fig. 4-2. Not all of these functions are needed for IJVM, but for the full JVM many of them would come in handy. In many cases, there are multiple possibilities for achieving the same result. In this table, + means arithmetic plus and – means arithmetic minus, so, for example  $-A$  means the two’s complement of  $A$ .

| $F_0$ | $F_1$ | ENA | ENB | INVA | INC | Function           |
|-------|-------|-----|-----|------|-----|--------------------|
| 0     | 1     | 1   | 0   | 0    | 0   | A                  |
| 0     | 1     | 0   | 1   | 0    | 0   | B                  |
| 0     | 1     | 1   | 0   | 1    | 0   | $\bar{A}$          |
| 1     | 0     | 1   | 1   | 0    | 0   | $\bar{B}$          |
| 1     | 1     | 1   | 1   | 0    | 0   | $A + B$            |
| 1     | 1     | 1   | 1   | 0    | 1   | $A + B + 1$        |
| 1     | 1     | 1   | 0   | 0    | 1   | $A + 1$            |
| 1     | 1     | 0   | 1   | 0    | 1   | $B + 1$            |
| 1     | 1     | 1   | 1   | 1    | 1   | $B - A$            |
| 1     | 1     | 0   | 1   | 1    | 0   | $B - 1$            |
| 1     | 1     | 1   | 0   | 1    | 1   | $-A$               |
| 0     | 0     | 1   | 1   | 0    | 0   | $A \text{ AND } B$ |
| 0     | 1     | 1   | 1   | 0    | 0   | $A \text{ OR } B$  |
| 0     | 1     | 0   | 0   | 0    | 0   | 0                  |
| 1     | 1     | 0   | 0   | 0    | 1   | 1                  |
| 1     | 1     | 0   | 0   | 1    | 0   | $-1$               |

**Figure 4-2.** Useful combinations of ALU signals and the function performed.

The ALU of Fig. 4-1 needs two data inputs: a left input (A) and a right input (B). Attached to the left input is a holding register, H. Attached to the right input is the B bus, which can be loaded from any one of nine sources, indicated by the nine gray arrows touching it. An alternative design, with two full buses, has a different set of trade-offs and will be discussed later in this chapter.

H can be loaded by choosing an ALU function that just passes the right input (from the B bus) through to the ALU output. One such function is adding the ALU inputs, only with ENA negated so the left input is forced to zero. Adding zero to the value on the B bus just yields the value on the B bus. This result can then be passed through the shifter unmodified and stored in H.

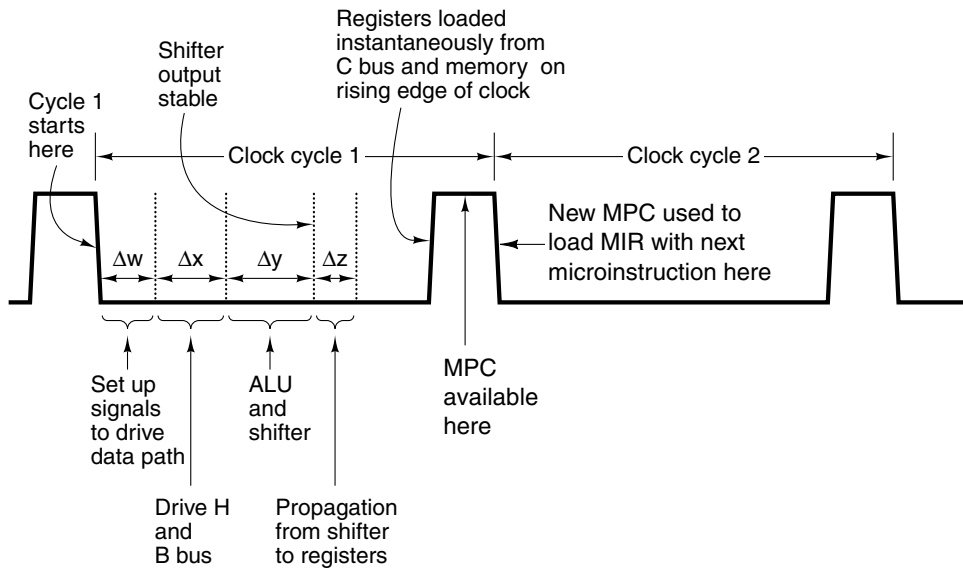
In addition to the above functions, two other control lines can be used independently to control the output from the ALU. SLL8 (Shift Left Logical) shifts the contents left by 1 byte, filling the 8 least significant bits with zeros. SRA1 (Shift Right Arithmetic) shifts the contents right by 1 bit, leaving the most significant bit unchanged.

It is explicitly possible to read and write the same register on one cycle. For example, it is allowed to put SP onto the B bus, disable the ALU's left input, enable the INC signal, and store the result in SP, thus incrementing SP by 1 (see the eighth line in Fig. 4-2). How can a register be read and written on the same cycle without producing garbage? The solution is that reading and writing are actually performed at different times within the cycle. When a register is selected as the ALU's right input, its value is put onto the B bus early in the cycle and kept there continuously throughout the entire cycle. The ALU then does its work, producing a result that passes through the shifter onto the C bus. Near the end of the cycle, when the ALU and shifter outputs are known to be stable, a clock signal triggers the store of the contents of the C bus into one or more of the registers. One of these registers may well be the one that supplied the B bus with its input. The precise timing of the data path makes it possible to read and write the same register on one cycle, as described below.

### Data Path Timing

The timing of these events is shown in Fig. 4-3. Here a short pulse is produced at the start of each clock cycle. It can be derived from the main clock, as shown in Fig. 3-21(c). On the falling edge of the pulse, the bits that will drive all the gates are set up. This takes a finite and known time,  $\Delta w$ . Then the register needed on the B bus is selected and driven onto the B bus. It takes  $\Delta x$  before the value is stable. Then the ALU and shifter begin to operate on valid data. After another  $\Delta y$ , the ALU and shifter outputs are stable. After an additional  $\Delta z$ , the results have propagated along the C bus to the registers, where they can be loaded on the rising edge of the next pulse. The load should be edge triggered and fast, so that even if some of the input registers are changed, the effects will not be felt on the C bus until long after the registers have been loaded. Also on the rising edge of the pulse, the register driving the B bus stops doing so, in preparation for the next cycle. MPC, MIR, and the memory are mentioned in the figure; their roles will be discussed shortly.

It is important to realize that even though there are no storage elements in the data path, there is a finite propagation time through it. Changing the value on the



**Figure 4-3.** Timing diagram of one data path cycle.

B bus does not cause the C bus to change until a finite time later (due to the finite delays of each step). Consequently, even if a store changes one of the input registers, the value will be safely tucked away in the register long before the (now-incorrect) value being put on the B bus (or H) can reach the ALU.

Making this design work requires rigid timing, a long clock cycle, a known minimum propagation time through the ALU, and a fast load of the registers from the C bus. However, with careful engineering, the data path can be designed so that it functions correctly all the time. Actual machines work this way.

A somewhat different way to look at the data path cycle is to think of it as broken up into implicit subcycles. The start of subcycle 1 is triggered by the falling edge of the clock. The activities that go on during the subcycles are shown below along with the subcycle lengths (in parentheses).

1. The control signals are set up ( $\Delta w$ ).
2. The registers are loaded onto the B bus ( $\Delta x$ ).
3. The ALU and shifter operate ( $\Delta y$ ).
4. The results propagate along the C bus back to the registers ( $\Delta z$ ).

At the rising edge of the next clock cycle, the results are stored in the registers.

We said that the subcycles can be best thought of as being *implicit*. By this we mean there are no clock pulses or other explicit signals to tell the ALU when

to operate or tell the results to enter the C bus. In reality, the ALU and shifter run all the time. However, their inputs are garbage until a time  $\Delta w + \Delta x$  after the falling edge of the clock. Likewise, their outputs are garbage until  $\Delta w + \Delta x + \Delta y$  has elapsed after the falling edge of the clock. The only explicit signals that drive the data path are the falling edge of the clock, which starts the data path cycle, and the rising edge of the clock, which loads the registers from the C bus. The other subcycle boundaries are implicitly determined by the inherent propagation times of the circuits involved. It is the responsibility of the design engineers to make sure that the time  $\Delta w + \Delta x + \Delta y + \Delta z$  comes sufficiently in advance of the rising edge of the clock to have the register loads work all the time.

## Memory Operation

Our machine has two different ways to communicate with memory: a 32-bit, word-addressable memory port and an 8-bit, byte-addressable memory port. The 32-bit port is controlled by two registers, MAR (**Memory Address Register**), and MDR (**Memory Data Register**), as shown in Fig. 4-1. The 8-bit port is controlled by one register, PC, which reads 1 byte into the low-order 8 bits of MBR. This port can only read data from memory; it cannot write data to memory.

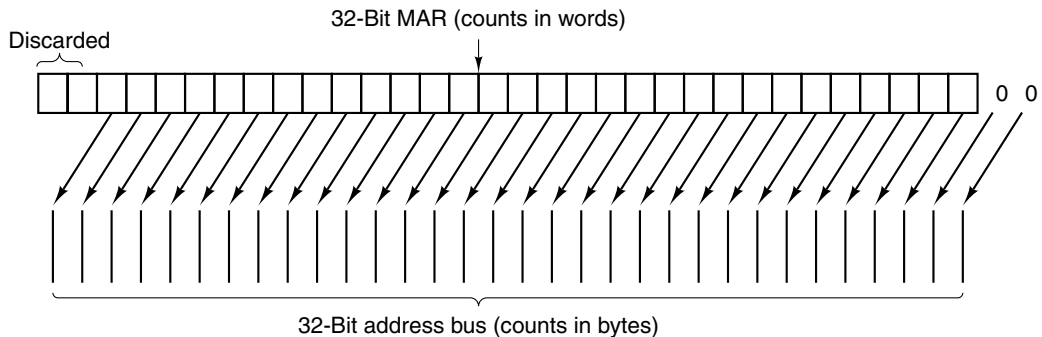
Each of these registers (and all the other registers in Fig. 4-1) are driven by one or two **control signals**. An open arrow under a register indicates a control signal that enables the register's output onto the B bus. Since MAR does not have a connection to the B bus, it does not have an enable signal. H does not have one either because it is always enabled, it being the only possible left ALU input.

A solid black arrow under a register indicates a control signal that writes (i.e., loads) the register from the C bus. Since MBR cannot be loaded from the C bus, it does not have a write signal (although it does have two other enable signals, described below). To initiate a memory read or write, the appropriate memory registers must be loaded, then a read or write signal issued to the memory (not shown in Fig. 4-1).

MAR contains *word* addresses, so that the values 0, 1, 2, etc., refer to consecutive words. PC contains *byte* addresses, so that the values 0, 1, 2, etc. refer to consecutive bytes. Thus putting a 2 in PC and starting a memory read will read out byte 2 from memory and put it in the low-order 8 bits of MBR. Putting a 2 in MAR and starting a memory read will read out bytes 8–11 (i.e., word 2) from memory and put them in MDR.

This difference in functionality is needed because MAR and PC will be used to reference two different parts of memory. The need for this distinction will become clearer later. For the moment, suffice it to say that the MAR/MDR combination is used to read and write ISA-level data words and the PC/MBR combination is used to read the executable ISA-level program, which consists of a byte stream. All other registers that contain addresses use word addresses, like MAR.

In the actual physical implementation, there is only one real memory and it is byte oriented. Allowing MAR to count in words (needed due to the way JVM is defined) while the physical memory counts in bytes is handled by a simple trick. When MAR is placed on the address bus, its 32 bits do not map onto the 32 address lines, 0 – 31, directly. Instead MAR bit 0 is wired to address bus line 2, MAR bit 1 is wired to address bus line 3, and so on. The upper 2 bits of MAR are discarded since they are only needed for word addresses above  $2^{32}$ , none of which are legal for our 4-GB machine. Using this mapping, when MAR is 1, address 4 is put onto the bus; when MAR is 2, address 8 is put onto the bus, and so forth. This trick is illustrated in Fig. 4-4.



**Figure 4-4.** Mapping of the bits in MAR to the address bus.

As mentioned above, data read from memory through the 8-bit memory port are returned in MBR, an 8-bit register. MBR can be gated (i.e., copied) onto the B bus in one of two ways: unsigned or signed. When the unsigned value is needed, the 32-bit word put onto the B bus contains the MBR value in the low-order 8 bits and zeros in the upper 24 bits. Unsigned values are useful for indexing into a table, or when a 16-bit integer has to be assembled from 2 consecutive (unsigned) bytes in the instruction stream.

The other option for converting the 8-bit MBR to a 32-bit word is to treat it as a signed value between  $-128$  and  $+127$  and use this value to generate a 32-bit word with the same numerical value. This conversion is done by duplicating the MBR sign bit (leftmost bit) into the upper 24 bit positions of the B bus, a process known as **sign extension**. When this option is chosen, the upper 24 bits will either be all 0s or all 1s, depending on whether the leftmost bit of the 8-bit MBR is a 0 or a 1.

The choice of whether the 8-bit MBR is converted to an unsigned or a signed 32-bit value on the B bus is determined by which of the two control signals (open arrows below MBR in Fig. 4-1) is asserted. The need for these two options is why two arrows are present. The ability to have the 8-bit MBR act like a 32-bit source to the B bus is indicated by the dashed box to the left of MBR in the figure.



### 4.1.2 Microinstructions

To control the data path of Fig. 4-1, we need 29 signals. These can be divided into five functional groups, as described below.

- 9 Signals to control writing data from the C bus into registers.
- 9 Signals to control enabling registers onto the B bus for ALU input.
- 8 Signals to control the ALU and shifter functions.
- 2 Signals (not shown) to indicate memory read/write via MAR/MDR.
- 1 Signal (not shown) to indicate memory fetch via PC/MBR.

The values of these 29 control signals specify the operations for one cycle of the data path. A cycle consists of gating values out of registers and onto the B bus, propagating the signals through the ALU and shifter, driving them onto the C bus, and finally writing the results in the appropriate register or registers. In addition, if a memory read data signal is asserted, the memory operation is started at the end of the data path cycle, after MAR has been loaded. The memory data are available at the very end of the *following* cycle in MBR or MDR and can be used in the cycle *after that*. In other words, a memory read on either port initiated at the end of cycle  $k$  delivers data that cannot be used in cycle  $k + 1$ , but only in cycle  $k + 2$  or later.

This seemingly counterintuitive behavior is explained by Fig. 4-3. The memory control signals are not generated in clock cycle 1 until just after MAR and PC are loaded at the rising edge of the clock, toward the end of clock cycle 1. We will assume the memory puts its results on the memory buses within one cycle so that MBR and/or MDR can be loaded on the next rising clock edge, along with the other registers.

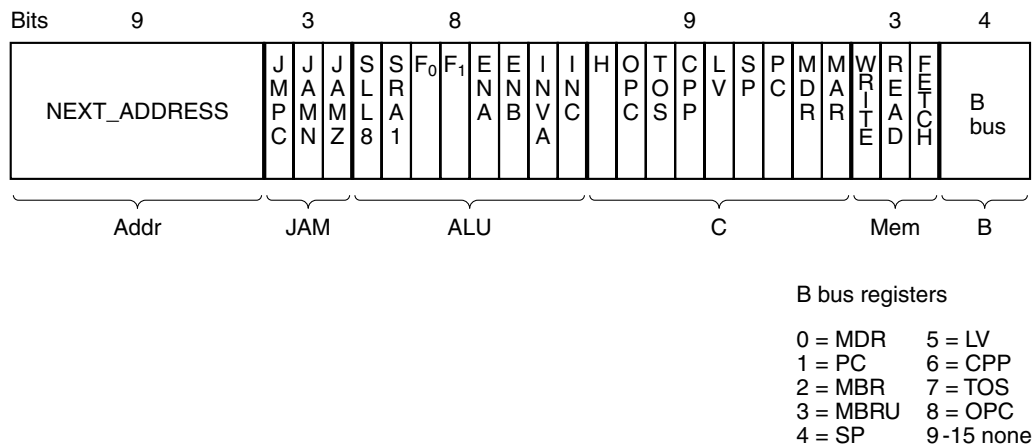
Put in other words, we load MAR at the end of a data path cycle and start the memory shortly thereafter. Consequently, we cannot really expect the results of a read operation to be in MDR at the start of the next cycle, especially if the width of the clock pulse is short. There is just not enough time if the memory takes one clock cycle. One data path cycle must intervene between starting a memory read and using the result. Of course, other operations can be performed during that cycle, just not ones that need the memory word.

The assumption that the memory takes one cycle to operate is equivalent to assuming that the level 1 cache hit rate is 100%. This assumption is never true, but the complexity introduced by a variable-length memory cycle time is more than we want to deal with here.

Since MBR and MDR are loaded on the rising edge of the clock, along with all the other registers, they may be read during cycles when a new memory read is being performed. They return the old values since the read has not yet had time to

overwrite them. There is no ambiguity here; until new values are loaded into MBR and MDR at the rising edge of the clock, the previous values are still there and usable. Note that it is possible to perform back-to-back reads on two consecutive cycles since a read only takes 1 cycle. Also, both memories may operate at the same time. However, trying to read and write the same byte simultaneously gives undefined results.

While it may be desirable to write the output on the C bus into more than one register, it is never desirable to enable more than one register onto the B bus at a time. (In fact, some real implementations will suffer physical damage if this is done.) With a small increase in circuitry, we can reduce the number of bits needed to select among the possible sources for driving the B bus. There are only nine possible input registers that can drive the B bus (where the signed and unsigned versions of MBR each count separately). Therefore, we can encode the B bus information in 4 bits and use a decoder to generate the 16 control signals, 7 of which are not needed. In a commercial design, the architects would experience an overwhelming urge to get rid of one of the registers so that 3 bits would do the job. As academics, we have the enormous luxury of being able to waste 1 bit to give a cleaner and simpler design.



**Figure 4-5.** The microinstruction format for the Mic-1.

At this point we can control the data path with  $9 + 4 + 8 + 2 + 1 = 24$  signals, hence 24 bits. However, these 24 bits only control the data path for one cycle. The second part of the control is to determine what is to be done on the following cycle. To include this in the design of the controller, we will create a format for describing the operations to be performed using the 24 control bits plus two additional fields: the NEXT\_ADDRESS field and the JAM field. The contents of each of these fields will be discussed shortly. Figure 4-5 shows a possible format, divided into the six groups, and containing the following 36 signals:

- Addr – Contains the address of a potential next microinstruction.
- JAM – Determines how the next microinstruction is selected.
- ALU – ALU and shifter functions.
- C – Selects which registers are written from the C bus.
- Mem – Memory functions.
- B – Selects the B bus source; it is encoded as shown.

The ordering of the groups is, in principle, arbitrary although we have actually chosen it very carefully to minimize line crossings in Fig. 4-6. Line crossings in schematic diagrams like Fig. 4-6 often correspond to wire crossings on chips, which cause trouble in two-dimensional designs and are best minimized.

### 4.1.3 Microinstruction Control: The Mic-1

So far we have described how the data path is controlled, but we have not yet described how it is decided which of the control signals should be enabled on each cycle. This is determined by a **sequencer** that is responsible for stepping through the sequence of operations necessary for the execution of a single ISA instruction.

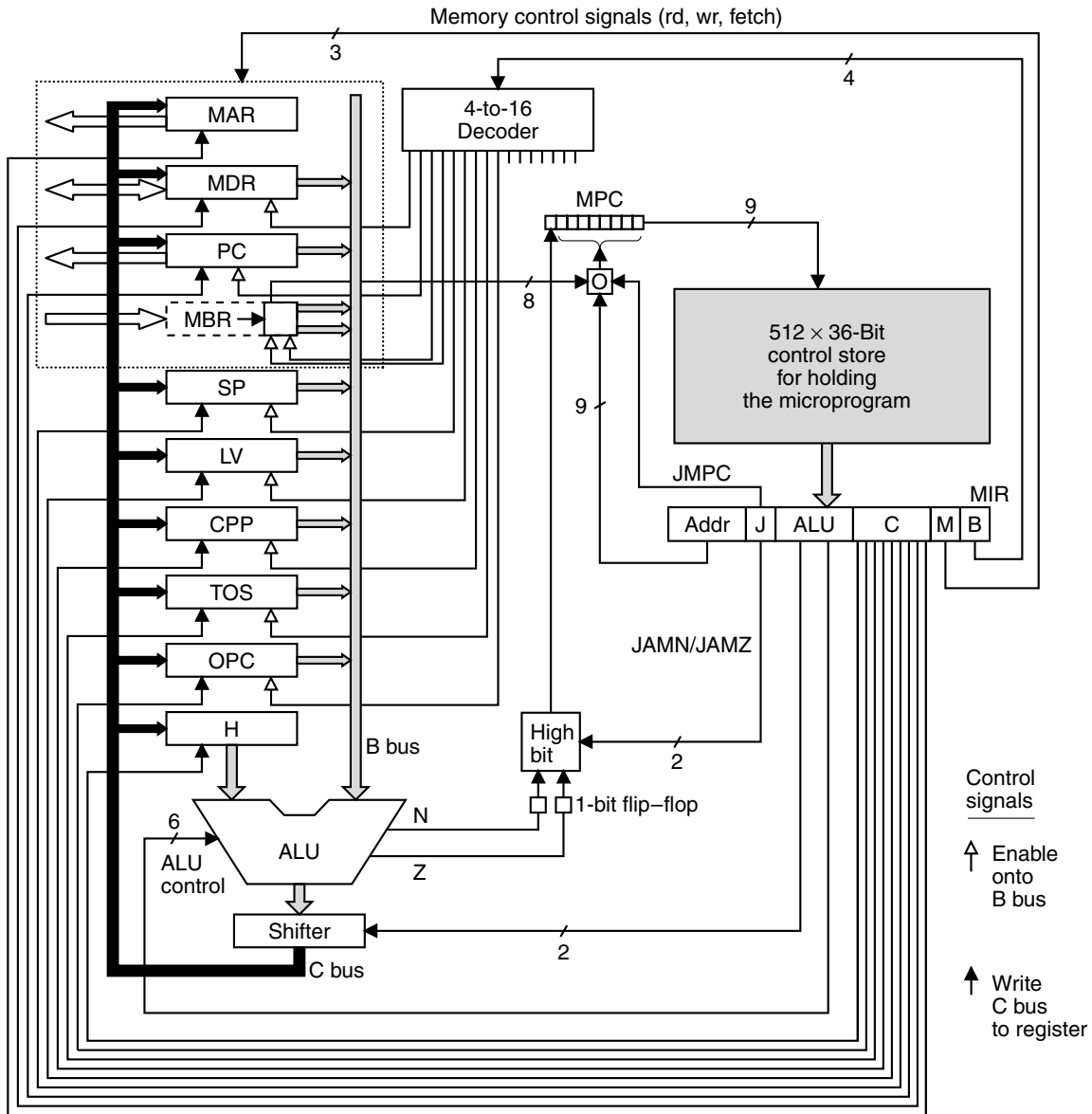
The sequencer must produce two kinds of information each cycle:

1. The state of every control signal in the system.
2. The address of the microinstruction that is to be executed next.

Figure 4-6 is a detailed block diagram of the complete microarchitecture of our example machine, which we will call the **Mic-1**. It may look imposing initially but it is worth studying carefully. When you fully understand every box and every line in this figure, you will be well on your way to understanding the microarchitecture level. The block diagram has two parts: the data path, on the left, which we have already discussed in detail, and the control section, on the right, which we will now look at.

The largest and most important item in the control portion of the machine is a memory called the **control store**. It is convenient to think of it as a memory that holds the complete microprogram, although it is sometimes implemented as a set of logic gates. In general, we will refer to it as the control store, to avoid confusion with the main memory, accessed through MBR and MDR. However, functionally, the control store is a memory that simply holds microinstructions instead of ISA instructions. For our example machine, it contains 512 words, each word consisting of one 36-bit microinstruction of the kind illustrated in Fig. 4-5. Actually, not all of these words are needed, but (for reasons to be explained shortly) we need addresses for 512 distinct words.

In one important way, the control store is quite different from the main memory: instructions in main memory are always executed in address order (except for



**Figure 4-6.** The complete block diagram of our example microarchitecture, the Mic-1.

branches); microinstructions are not. The act of incrementing the program counter in Fig. 2-3 expresses the fact that the default instruction to execute after the current one is the instruction following the current one in memory. Microprograms need more flexibility (because microinstruction sequences tend to be short), so they usually do not have this property. Instead, each microinstruction explicitly specifies its successor.

Since the control store is functionally a (read-only) memory, it needs its own memory address register and its own memory data register. It does not need read and write signals, because it is continuously being read. We will call the control store's memory address register **MPC (MicroProgram Counter)**. This name is ironic since the locations in it are explicitly not ordered, so the concept of counting is not useful (but who are we to argue with tradition?). The memory data register is called **MIR (MicroInstruction Register)**. Its function is to hold the current microinstruction, whose bits drive the control signals that operate the data path.

The MIR register in Fig. 4-6 holds the same six groups as Fig. 4-5. The Addr and J (for JAM) groups control the selection of the next microinstruction and will be discussed shortly. The ALU group contains the 8 bits that select the ALU function and drive the shifter. The C bits cause individual registers to load the ALU output from the C bus. The M bits control memory operations.

Finally, the last 4 bits drive the decoder that determines what goes onto the B bus. In this case we have chosen to use a standard 4-to-16 decoder, even though only nine possibilities are required. In a more finely-tuned design, a 4-to-9 decoder could be used. The trade-off here is using a standard circuit taken from a library of circuits versus designing a custom one. Using the standard circuit is simpler and is unlikely to introduce any bugs. Rolling your own uses less chip area but takes longer to design and you might get it wrong.

The operation of Fig. 4-6 is as follows. At the start of each clock cycle (the falling edge of the clock in Fig. 4-3), MIR is loaded from the word in the control store pointed to by MPC. The MIR load time is indicated in the figure by  $\Delta w$ . If one thinks in terms of subcycles, MIR is loaded during the first one.

Once the microinstruction is set up in MIR, the various signals propagate out into the data path. A register is put out onto the B bus, the ALU knows which operation to perform, and there is lots of activity out there. This is the second subcycle. After an interval  $\Delta w + \Delta x$  from the start of the cycle, the ALU inputs are stable.

Another  $\Delta y$  later, everything has settled down and the ALU, N, Z, and shifter outputs are stable. The N and Z values are then saved in a pair of 1-bit flip-flops. These bits, like all the registers that are loaded from the C bus and from memory, are saved on the rising edge of the clock, near the end of the data path cycle. The ALU output is not latched but just fed into the shifter. The ALU and shifter activity occurs during subcycle 3.

After an additional interval,  $\Delta z$ , the shifter output has reached the registers via the C bus. Then the registers can be loaded near the end of the cycle (at the rising edge of the clock pulse in Fig. 4-3). Subcycle 4 consists of loading the registers and N and Z flip-flops. It terminates a little after the rising edge of the clock, when all the results have been saved and the results of the previous memory operations are available and MPC has been loaded. This process goes on and on until somebody gets bored with it and turns the machine off.

In parallel with driving the data path, the microprogram has to determine which microinstruction to execute next, as they need not be run in the order they appear in the control store. The calculation of the address of the next microinstruction begins after MIR has been loaded and is stable. First, the 9-bit NEXT\_ADDRESS field is copied to MPC. While this copy is taking place, the JAM field is inspected. If it has the value 000, nothing else is done; when the copy of NEXT\_ADDRESS completes, MPC will point to the next microinstruction.

If one or more of the JAM bits are 1, more work is needed. If JAMN is set, the 1-bit N flip-flop is ORed into the high-order bit of MPC. Similarly, if JAMZ is set, the 1-bit Z flip-flop is ORed there. If both are set, both are ORed there. The reason that the N and Z flip-flops are needed is that after the rising edge of the clock (while the clock is high), the B bus is no longer being driven, so the ALU outputs can no longer be assumed to be correct. Saving the ALU status flags in N and Z makes the correct values available and stable for the MPC computation, no matter what is going on around the ALU.

In Fig. 4-6, the logic that does this computation is labeled “High bit.” The Boolean function it computes is

$$F = (\text{JAMZ AND Z}) \text{ OR } (\text{JAMN AND N}) \text{ OR NEXT\_ADDRESS}[8]$$

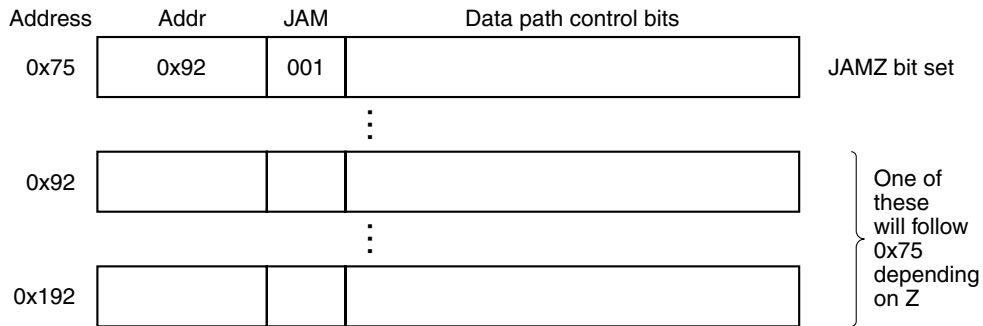
Note that in all cases, MPC can take on only one of two possible values:

1. The value of NEXT\_ADDRESS.
2. The value of NEXT\_ADDRESS with the high-order bit ORed with 1.

No other possibilities exist. If the high-order bit of NEXT\_ADDRESS was already 1, using JAMN or JAMZ makes no sense.

Note that when the JAM bits are all zeros, the address of the next microinstruction to be executed is simply the 9-bit number in its NEXT\_ADDRESS field. When either JAMN or JAMZ are 1, there are two potential successors: NEXT\_ADDRESS and NEXT\_ADDRESS ORed with 0x100 (assuming that NEXT\_ADDRESS ≤ 0xFF). (Note that 0x indicates that the number following it is in hexadecimal.) This point is illustrated in Fig. 4-7. The current microinstruction, at location 0x75, has NEXT\_ADDRESS = 0x92 and JAMZ set to 1. Consequently, the next address of the microinstruction depends on the Z bit stored on the previous ALU operation. If the Z bit is 0, the next microinstruction comes from 0x92. If the Z bit is 1, the next microinstruction comes from 0x192.

The third bit in the JAM field is JMPN. If it is set, the 8 MBR bits are bitwise ORed with the 8 low-order bits of the NEXT\_ADDRESS field coming from the current microinstruction. The result is sent to MPC. The box with the label “O” in Fig. 4-6 does an OR of MBR with NEXT\_ADDRESS if JMPN is 1 but just passes NEXT\_ADDRESS through to MPC if JMPN is 0. When JMPN is 1, the low-order 8 bits of NEXT\_ADDRESS are normally zero. The high-order bit can be 0 or 1, so the NEXT\_ADDRESS value used with JMPN is normally 0x000 or 0x100. The reason for sometimes using 0x000 and sometimes using 0x100 will be discussed later.



**Figure 4-7.** A microinstruction with JAMZ set to 1 has two potential successors.

The ability to OR MBR together with NEXT\_ADDRESS and store the result in MPC allows an efficient implementation of a multiway branch (jump). Notice that any of 256 addresses can be specified, determined solely by the bits present in MBR. In a typical use, MBR contains an opcode, so the use of JMPZ will result in a unique selection for the next microinstruction to be executed for every possible opcode. This method is useful for quickly branching directly to the function corresponding to the just-fetched opcode.

Understanding the timing of the machine is critical to what will follow, so it is perhaps worth repeating it again. We will do it in terms of subcycles, since this is easy to visualize, but the only real clock events are the falling edge, which starts the cycle, and the rising edge, which loads the registers and the N and Z flip-flops. Please refer to Fig. 4-3 once more.

During subcycle 1, initiated by the falling edge of the clock, MIR is loaded from the address currently held in MPC. During subcycle 2, the signals from MIR propagate out and the B bus is loaded from the selected register. During subcycle 3, the ALU and shifter operate and produce a stable result. During subcycle 4, the C bus, memory buses, and ALU values become stable. At the rising edge of the clock, the registers are loaded from the C bus, N and Z flip-flops are loaded, and MBR and MDR get their results from the memory operation started at the end of the previous data path cycle (if any). As soon as MBR is available, MPC is loaded in preparation for the next microinstruction. Thus MPC gets its value sometime during the middle of the interval when the clock is high but after MBR/MDR are ready. It could be either level triggered (rather than edge triggered), or edge trigger a fixed delay after the rising edge of the clock. All that matters is that MPC is not loaded until the registers it depends on (MBR, N, and Z) are ready. As soon as the clock falls, MPC can address the control store and a new cycle can begin.

Note that each cycle is self contained. It specifies what goes onto the B bus, what the ALU and shifter are to do, where the C bus is to be stored, and finally, what the next MPC value should be.

One final note about Fig. 4-6 is worth making. We have been treating MPC as a proper register, with 9 bits of storage capacity that is loaded while the clock is

high. In reality, there is no need to have a register there at all. All of its inputs can be fed directly through to the control store. As long as they are present at the control store at the falling edge of the clock when MIR is selected and read out, that is sufficient. There is no need to actually store them in MPC. For this reason, MPC might well be implemented as a **virtual register**, which is just a gathering place for signals, more like an electronic patch panel, than a real register. Making MPC a virtual register simplifies the timing: now events only happen on the falling and rising edges of the clock and nowhere else. But if it is easier for you to think of MPC as a real register, that is also a valid viewpoint.

## 4.2 AN EXAMPLE ISA: IJVM

Let us continue our example by introducing the ISA level of the machine to be interpreted by the microprogram running on the microarchitecture of Fig. 4-6 (IJVM). For convenience, we will sometimes refer to the Instruction Set Architecture as the **macroarchitecture**, to contrast it with the microarchitecture. Before we describe IJVM, however, we will digress slightly to motivate it.

### 4.2.1 Stacks

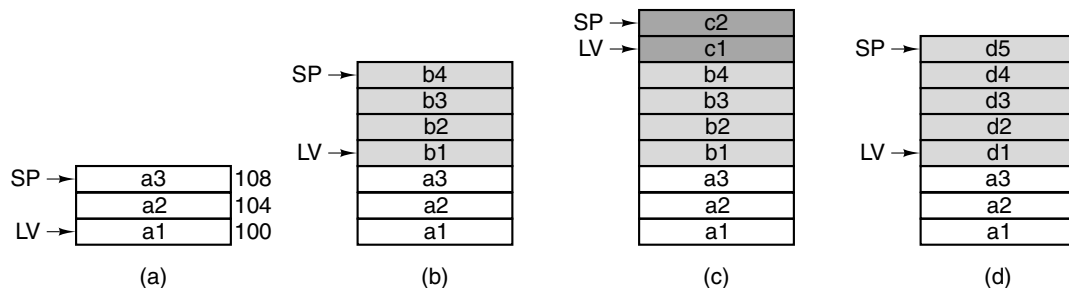
Virtually all programming languages support the concept of procedures (methods), which have local variables. These variables can be accessed from inside the procedure but cease to be accessible once the procedure has returned. The question thus arises: “Where should these variables be kept in memory?”

The simplest solution, to give each variable an absolute memory address, does not work. The problem is that a procedure may call itself. We will study these recursive procedures in Chap. 5. For the moment, suffice it to say that if a procedure is active (i.e., called) twice, it is impossible to store its variables in absolute memory locations because the second invocation will interfere with the first.

Instead, a different strategy is used. An area of memory, called the **stack**, is reserved for variables, but individual variables do not get absolute addresses in it. Instead, a register, say, LV, is set to point to the base of the local variables for the current procedure. In Fig. 4-8(a), a procedure *A*, which has local variables *a1*, *a2*, and *a3*, has been called, so storage for its local variables has been reserved starting at the memory location pointed to by LV. Another register, SP, points to the highest word of *A*’s local variables. If LV is 100 and words are 4 bytes, then SP will be 108. Variables are referred to by giving their offset (distance) from LV. The data structure between LV and SP (and including both words pointed to) is called *A*’s **local variable frame**.

Now let us consider what happens if *A* calls another procedure, *B*. Where should *B*’s four local variables (*b1*, *b2*, *b3*, *b4*) be stored? Answer: On the stack, on top of *A*’s, as shown in Fig. 4-8(b). Notice that LV has been adjusted by the





**Figure 4-8.** Use of a stack for storing local variables. (a) While *A* is active. (b) After *A* calls *B*. (c) After *B* calls *C*. (d) After *C* and *B* return and *A* calls *D*.

procedure call to point to *B*'s local variables instead of *A*'s. *B*'s local variables can be referred to by giving their offset from LV. Similarly, if *B* calls *C*, LV and SP are adjusted again to allocate space for *C*'s two variables, as shown in Fig. 4-8(c).

When *C* returns, *B* becomes active again, and the stack is adjusted back to Fig. 4-8(b) so that LV now points to *B*'s local variables again. Likewise, when *B* returns, we get back to the situation of Fig. 4-8(a). Under all conditions, LV points to the base of the stack frame for the currently active procedure, and SP points to the top of the stack frame.

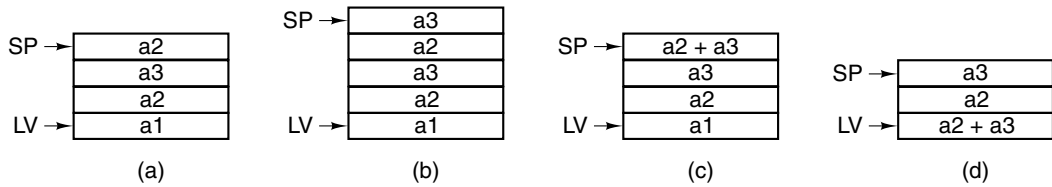
Now suppose that *A* calls *D*, which has five local variables. We get the situation of Fig. 4-8(d), in which *D*'s local variables use the same memory that *B*'s did, as well as part of *C*'s. With this memory organization, memory is only allocated for procedures that are currently active. When a procedure returns, the memory used by its local variables is released.

Stacks have another use, in addition to holding local variables. They can be used for holding operands during the computation of an arithmetic expression. When used this way, the stack is referred to as the **operand stack**. Suppose, for example, that before calling *B*, *A* has to do the computation

$$a1 = a2 + a3;$$

One way of doing this sum is to push *a2* onto the stack, as shown in Fig. 4-9(a). Here SP has been incremented by the number of bytes in a word, say, 4, and the first operand stored at the address now pointed to by SP. Next, *a3* is pushed onto the stack, as shown in Fig. 4-9(b). As an aside on notation, we will typeset all program fragments in Helvetica, as above. We will also use this font for assembly language opcodes and machine registers, but in running text, program variables and procedures will be given in *italics*. The difference is that variables and procedure names are chosen by the user; opcodes and register names are built in.

The actual computation can be done by now executing an instruction that pops two words off the stack, adds them together, and pushes the result back onto the stack, as shown in Fig. 4-9(c). Finally, the top word can be popped off the stack and stored back in local variable *a1*, as illustrated in Fig. 4-9(d).



**Figure 4-9.** Use of an operand stack for doing an arithmetic computation.

The local variable frames and the operand stacks can be intermixed. For example, when computing an expression like  $x^2 + f(x)$  part of the expression (e.g.,  $x^2$ ) may be on the operand stack when a function  $f$  is called. The result of the function is left on the stack, on top of  $x^2$ , so the next instruction can add them.

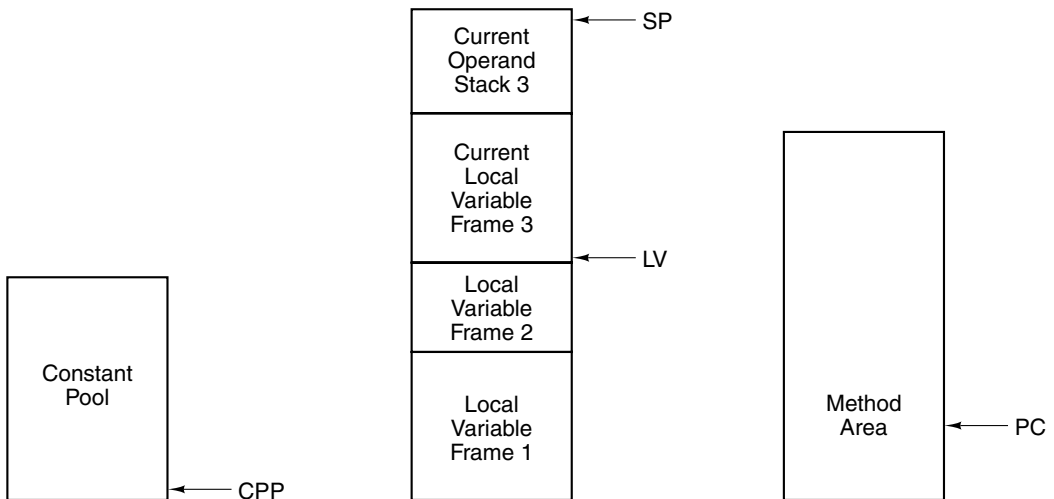
It is worth noting that while all machines use a stack for storing local variables, not all use an operand stack like this for doing arithmetic. In fact, most of them do not, but JVM and IJVM work like this, which is why we have introduced stack operations here. We will study them in more detail in Chap. 5.

### 4.2.2 The IJVM Memory Model

We are now ready to look at the IJVM's architecture. Basically, it consists of a memory that can be viewed in either of two ways: an array of 4,294,967,296 bytes (4 GB) or an array of 1,073,741,824 words, each consisting of 4 bytes. Unlike most ISAs, the Java Virtual Machine makes no absolute memory addresses directly visible at the ISA level, but there are several implicit addresses that provide the base for a pointer. IJVM instructions can only access memory by indexing from these pointers. At any time, the following areas of memory are defined:

1. *The Constant Pool.* This area cannot be written by an IJVM program and consists of constants, strings, and pointers to other areas of memory that can be referenced. It is loaded when the program is brought into memory and not changed afterward. There is an implicit register, CPP, that contains the address of the first word of the constant pool.
2. *The Local Variable Frame.* For each invocation of a method, an area is allocated for storing variables during the lifetime of the invocation. It is called the **local variable frame**. At the beginning of this frame reside the parameters (also called arguments) with which the method was invoked. The local variable frame does not include the operand stack, which is separate. However, for efficiency reasons, our implementation chooses to implement the operand stack immediately above the local variable frame. There is an implicit register that contains the address of the first location in the local variable frame. We will call this register LV. The parameters passed at the invocation of the method are stored at the beginning of the local variable frame.

3. *The Operand Stack.* The stack frame is guaranteed not to exceed a certain size, computed in advance by the Java compiler. The operand stack space is allocated directly above the local variable frame, as illustrated in Fig. 4-10. In our implementation, it is convenient to think of the operand stack as part of the local variable frame. In any case, there is an implicit register that contains the address of the top word of the stack. Notice that, unlike CPP and LV, this pointer, SP, changes during the execution of the method as operands are pushed onto the stack or popped from it.
4. *The Method Area.* Finally, there is a region of memory containing the program, referred to as the “text” area in a UNIX process. There is an implicit register that contains the address of the instruction to be fetched next. This pointer is referred to as the Program Counter, or PC. Unlike the other regions of memory, the Method Area is treated as a byte array.



**Figure 4-10.** The various parts of the IJVM memory.

One point needs to be made regarding the pointers. The CPP, LV, and SP registers are all pointers to *words*, not *bytes*, and are offset by the number of words. For the integer subset we have chosen, all references to items in the constant pool, the local variables frame, and the stack are words, and all offsets used to index into these frames are word offsets. For example, LV, LV + 1, and LV + 2 refer to the first three words of the local variables frame. In contrast, LV, LV + 4, and LV + 8 refer to words at intervals of four words (16 bytes).

In contrast, PC contains a byte address, and an addition or subtraction to PC changes the address by a number of bytes, not a number of words. Addressing for

PC is different from the others, and this fact is apparent in the special memory port provided for PC on the Mic-1. Remember that it is only 1 byte wide. Incrementing PC by one and initiating a read results in a fetch of the next *byte*. Incrementing SP by one and initiating a read results in a fetch of the next *word*.

### 4.2.3 The IJVM Instruction Set

The IJVM instruction set is shown in Fig. 4-11. Each instruction consists of an opcode and sometimes an operand, such as a memory offset or a constant. The first column gives the hexadecimal encoding of the instruction. The second gives its assembly language mnemonic. The third gives a brief description of its effect.

| Hex  | Mnemonic                  | Meaning                                                 |
|------|---------------------------|---------------------------------------------------------|
| 0x10 | BIPUSH <i>byte</i>        | Push byte onto stack                                    |
| 0x59 | DUP                       | Copy top word on stack and push onto stack              |
| 0xA7 | GOTO <i>offset</i>        | Unconditional branch                                    |
| 0x60 | IADD                      | Pop two words from stack; push their sum                |
| 0x7E | IAND                      | Pop two words from stack; push Boolean AND              |
| 0x99 | IFEQ <i>offset</i>        | Pop word from stack and branch if it is zero            |
| 0x9B | IFLT <i>offset</i>        | Pop word from stack and branch if it is less than zero  |
| 0x9F | IF_ICMPEQ <i>offset</i>   | Pop two words from stack; branch if equal               |
| 0x84 | IINC <i>varnum const</i>  | Add a constant to a local variable                      |
| 0x15 | ILOAD <i>varnum</i>       | Push local variable onto stack                          |
| 0xB6 | INVOKEVIRTUAL <i>disp</i> | Invoke a method                                         |
| 0x80 | IOR                       | Pop two words from stack; push Boolean OR               |
| 0xAC | IRETURN                   | Return from method with integer value                   |
| 0x36 | ISTORE <i>varnum</i>      | Pop word from stack and store in local variable         |
| 0x64 | ISUB                      | Pop two words from stack; push their difference         |
| 0x13 | LDC_W <i>index</i>        | Push constant from constant pool onto stack             |
| 0x00 | NOP                       | Do nothing                                              |
| 0x57 | POP                       | Delete word on top of stack                             |
| 0x5F | SWAP                      | Swap the two top words on the stack                     |
| 0xC4 | WIDE                      | Prefix instruction; next instruction has a 16-bit index |

**Figure 4-11.** The IJVM instruction set. The operands *byte*, *const*, and *varnum* are 1 byte. The operands *disp*, *index*, and *offset* are 2 bytes.

Instructions are provided to push a word from various sources onto the stack. These sources include the constant pool (LDC\_W), the local variable frame (ILOAD), and the instruction itself (BIPUSH). A variable can also be popped from